

AhamX Bodhi

Where I Becomes Infinite

What is Tokenization?

Module 3.3: Tokenization

Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- What tokenization is and why it exists
- How raw text becomes token IDs
- The concept of vocabulary and token types
- Why token boundaries matter for LLMs
- Key tokenization challenges in practice

Concept at a Glance

- **Duration:** 7 minutes
- **Bloom's Level:** B2 (Understand)
- **Difficulty:** L1 (Beginner)
- **Domain:** Foundational Theory
- **Module:** 3.3 Tokenization

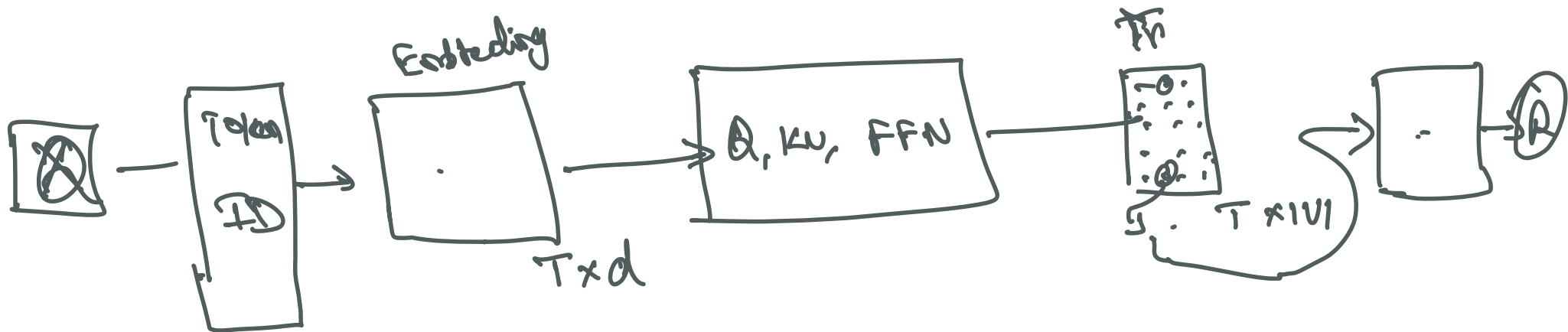


Learning Objectives

By the End of This Concept You Will Be Able To

- **Define** tokenization and explain its role in the LLM pipeline
- **Describe** how text is converted into token IDs using a vocabulary
- **Distinguish** between character, word, and subword tokenization strategies
- **Explain** why subword tokenization is preferred in modern LLMs
- **Identify** common tokenization edge cases: numbers, code, multilingual text
- **Connect** tokenization to downstream concepts: BPE, WordPiece, SentencePiece

Super-BPE





The Core Problem: Text vs Numbers

Neural Networks Only Understand Numbers

- A neural network processes tensors of floating point numbers
- Raw text is a sequence of characters — strings, not numbers
- Before any computation can occur, text must be converted to a numeric representation
- This conversion process is called **tokenization**

Tokenization is the **first and most fundamental step** in the LLM pipeline. Every word, sentence, and document you feed to GPT-4, Claude, or Llama must pass through a tokenizer before the model sees a single number.



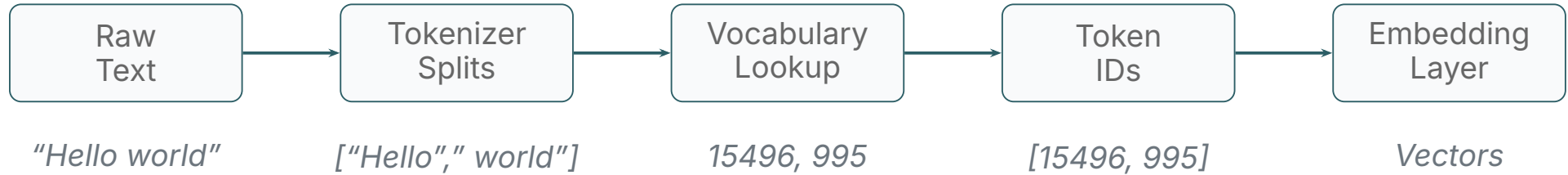
What is a Token?

Core Definition

- A **token** is the basic unit of text that a language model processes
- Tokens are **not** always words — they can be parts of words, punctuation, spaces, or even individual characters
- Each token in the vocabulary has a unique integer ID
- The model never sees raw text — it sees only a **sequence of token IDs**
- Example: the word `'tokenization'` might be split into `['token', 'ization']`



The Tokenization Pipeline



Every LLM starts with this pipeline. The tokenizer converts text into IDs; the embedding layer maps each ID to a dense vector the model can compute with.



Three Tokenization Strategies

Strategy Spectrum

- **Character-level:** Each character is a token. Small vocabulary, very long sequences
- **Word-level:** Each word is a token. Large vocabulary, out-of-vocabulary problem
- **Subword-level:** Words split into meaningful pieces. Best of both worlds

Why Subword Wins

- Handles unseen words gracefully
- Vocabulary size stays manageable (30K-100K)
- Captures morphological structure
- Works across multiple languages

Handwritten notes:
 Line reading . . . read
 Shuffled . . . study
 T5



Character-Level Tokenization

How It Works

- Every individual character (letter, digit, punctuation) becomes one token
- Example: ``cat'' → [``c``, ``a``, ``t``] → [67, 97, 116]
- Vocabulary is tiny: 26 letters + digits + symbols $\approx$ 100–300 tokens
- Sequences become very long: a 500-word document $\approx$ 2500 tokens

Character-level models handle any new word without failure, but the model must learn to compose meaning from individual letters — a much harder learning task. Modern LLMs do not use this approach for language.



Word-Level Tokenization

How It Works

- Each whitespace-delimited word is treated as one token
- Example: ``running fast'' → [``running'', ``fast''] → [4521, 892]
- Vocabulary can exceed 500K+ for a general corpus
- New words not in vocabulary get mapped to a special [UNK] token

The out-of-vocabulary (OOV) problem is severe: rare words, misspellings, new product names, and technical jargon all collapse to [UNK], destroying information. Word-level tokenization was used in early NLP but is now obsolete for LLMs.

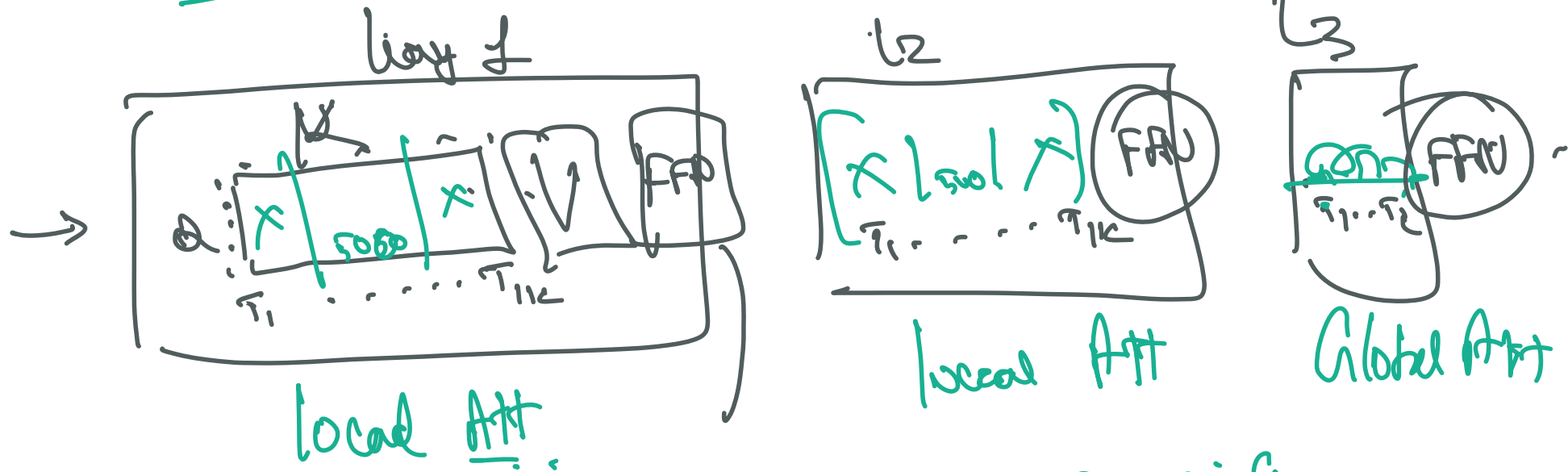


Subword Tokenization: The Modern Standard

Core Idea

- Common words are kept as single tokens: ``the'', ``is'', ``hello''
- Rare or long words are split into meaningful subword pieces
- Example: ``tokenization'' → [``token'', ``ization'']
- Example: ``unhappiness'' → [``un'', ``happy'', ``ness'']
- Vocabulary size is fixed and compact: typically 30,000 to 100,000 tokens
- **No OOV problem:** any word decomposes into known subwords

SWA: Sliding Window Attention



$L: L: H: 2: 2: G: 2: 2: G.$
SWA2: 1

The Vocabulary



What a Vocabulary Contains

- A fixed mapping from string tokens to integer IDs and back
- Built once during tokenizer training, then frozen for the model
- Includes special tokens: [PAD], [UNK], [BOS], [EOS], [MASK]
- GPT-2: 50,257 tokens GPT-4 / cl100k: 100,256 tokens Llama 3: 128,256 tokens

The vocabulary size directly affects the model embedding matrix size. A vocabulary of 100K tokens with embedding dimension 4096 produces a matrix of $100,000 \times 4096 = 409$ million parameters just for embeddings.



Special Tokens

Common Special Tokens

- **[BOS]** / **<s>** — Beginning of sequence
- **[EOS]** / **</s>** — End of sequence
- **[PAD]** — Padding for batch alignment
- **[UNK]** — Unknown token (word-level only)
- **[MASK]** — Masked token (BERT-style)

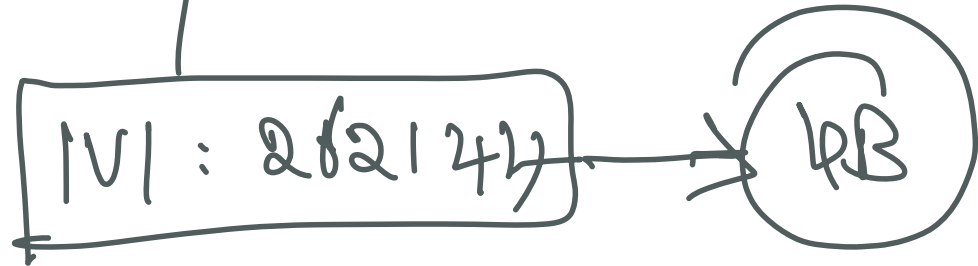
Chat Model Tokens

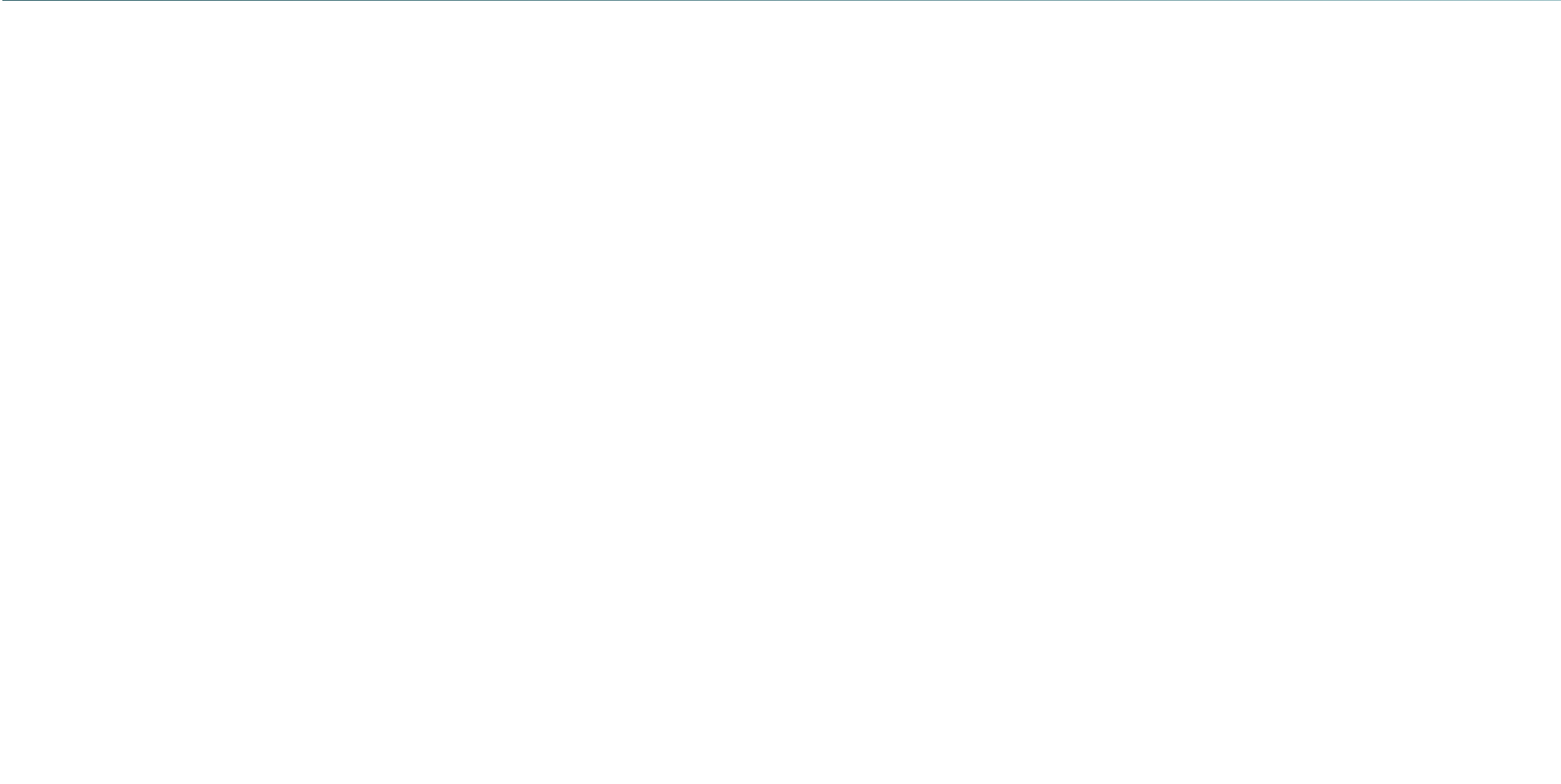
- **[INST]** — Instruction delimiter
- **<|im_start|>** — Turn start (GPT-4)
- **<|system|>** — System prompt marker
- These enable the model to distinguish roles in a conversation



Tokenization Strategy Comparison

Property	Character	Word	Subword	Byte-level
Vocab Size	~100	100K–500K	30K–128K	256
OOV Problem	None	Severe	None	None
Seq Length	Very long	Short	Medium	Longest
Multilingual	Good	Poor	Good	Excellent
Used In	Legacy	Legacy	GPT, BERT	GPT-2, Llama







Edge Case: Numbers and Arithmetic

Numbers Are Tokenized Inconsistently

- Small numbers may be single tokens: ``42'' → [42]
- Large numbers split arbitrarily: ``12345'' → [123, 45] or [1, 23, 45]
- The split depends entirely on training corpus frequency, not numeric value
- This is a known reason why LLMs struggle with arithmetic and digit counting

When GPT-4 makes arithmetic errors, tokenization is often a contributing factor. The model never “sees” the number 12345 as a unit — it sees fragments that do not have consistent numeric meaning.



Edge Cases: Code and Multilingual Text

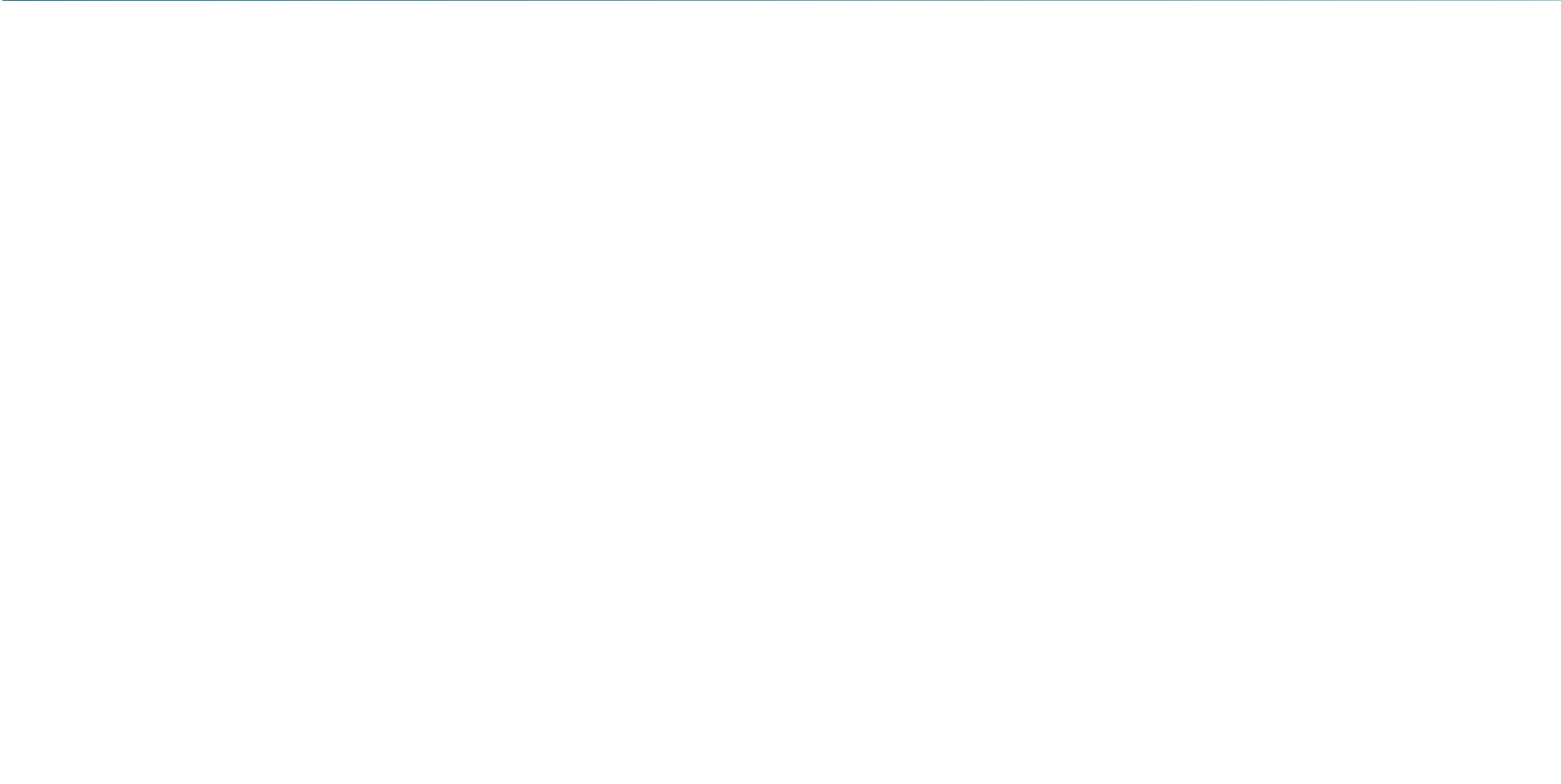
Code Tokenization

- Indentation spaces each become separate tokens
- A Python tab $\rightarrow$ 4 space tokens, inflating sequence length
- Keywords like `def`, `class`, `return` are usually single tokens
- Code-trained models (Codex, StarCoder) use code-optimized vocabularies

Non-English Text

- English-trained tokenizers are inefficient for other languages
- A single Hindi word may cost 5–10 tokens
- This makes non-English text more expensive (more tokens per idea)
- Multilingual models use larger vocabularies to compensate

$\text{Tokenization} = \left| \text{vocab} \right| \leftarrow \text{Tokenizer dat} \subseteq \text{Pre-trained data}$





GenAI Application: Token Counting and API Billing

Tokens as the Unit of Cost

- OpenAI, Anthropic, and Google charge per token for API usage
- GPT-4 Turbo (2024): \$10 per million input tokens, \$30 per million output tokens
- Claude 3 Opus: \$15 per million input tokens
- A typical business email (~200 words) costs approximately 270 tokens
- Prompt engineers optimize prompts to reduce token count and cut costs

Understanding tokenization is essential for cost forecasting in production GenAI systems. A single RAG pipeline with a 2K-token prompt, 100K daily requests, costs \$2,000/day at \$10/M tokens — small token savings have large financial impact.



GenAI Application: Context Window Management

Context Windows are Measured in Tokens

- GPT-4 Turbo supports 128K token context; Claude 3 supports 200K tokens
- Every character in the system prompt, conversation history, and RAG chunks consumes context space
- Developers must budget tokens: system prompt + history + retrieval + response reserve
- LangChain and LlamaIndex include token counters to prevent context overflow
- Truncation strategies (sliding window, summarization) manage long conversations



GenAI Application: Token Efficiency in Prompts

Costly Prompt Patterns

- Verbose instructions repeated in every call
- Full conversation history included without summarization
- Large documents included verbatim instead of chunked
- Padding whitespace and unnecessary formatting

Optimization Strategies

- Use system prompt caching (Anthropic, OpenAI)
- Summarize old turns before including history
- Select relevant document chunks via retrieval
- Prefer dense, direct instructions over verbose prose



GenAI Application: Tokenization Artifacts in Model Output

When Tokenization Causes Visible Problems

- Models struggle to count letters: “How many r’s in strawberry?” fails because straw and berry are single tokens
- Reversing strings is hard: the model must reverse token order, not character order
- Unusual spellings tokenize differently, causing inconsistent model behaviour

Many apparent “reasoning failures” in LLMs trace back to tokenization. Understanding this helps practitioners write prompts that avoid triggering these failure modes — for example, spacing out letters:

s-t-r-a-w-b-e-r-r-y.

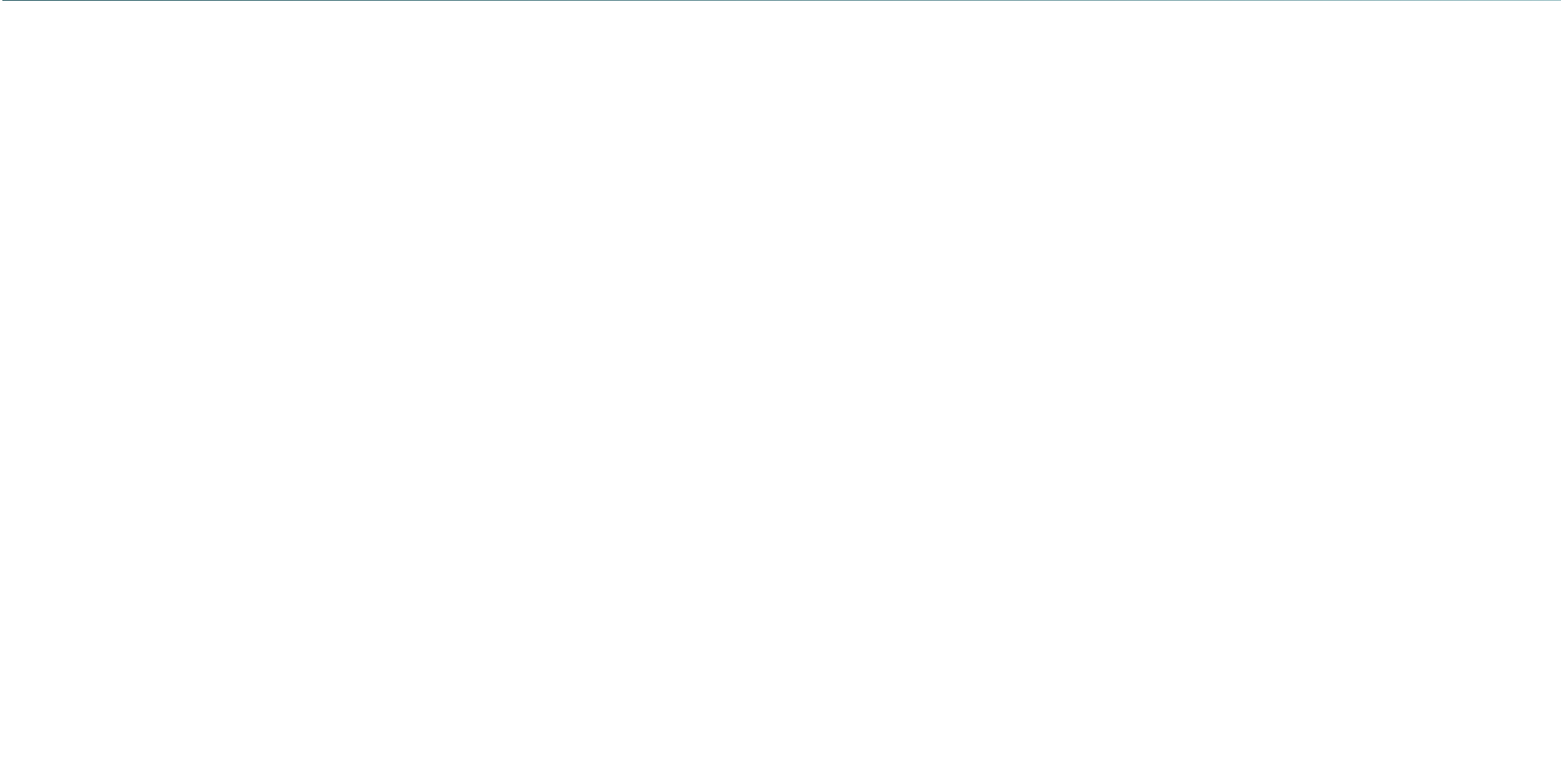


GenAI Application: Multilingual and Code Models

Tokenizer Design for Specialized Models

- **Llama 3 (Meta):** 128K vocabulary with multilingual coverage, reducing fertility for non-English text by 3x vs Llama 2
- **DeepSeek Coder:** 32K vocabulary enriched with programming keywords and identifier fragments
- **Gemini (Google):** Uses SentencePiece with byte-level fallback for any Unicode character
- **Phi-3 (Microsoft):** Uses tiktoken cl100k tokenizer; optimized for code reasoning tasks

Modern model families invest heavily in tokenizer design because an efficient tokenizer directly improves benchmark performance on multilingual and coding tasks without changing model architecture.





The Tokenization Algorithm Family

Three Dominant Algorithms

- **Byte Pair Encoding (BPE):** Iteratively merges the most frequent adjacent byte pairs. Used by GPT-2, GPT-4, Llama, Falcon
- **WordPiece:** Maximizes language model likelihood when merging. Used by BERT, DistilBERT, Electra
- **SentencePiece:** Language-agnostic, works directly on raw Unicode without pre-tokenization. Used by T5, Gemma, Llama 2, NLLB
- All three produce subword vocabularies; they differ in **how** the vocabulary is learned



Common Misconceptions

Myths About Tokenization

- **Myth:** "One token = one word." In reality, common words are one token; rare words may be 3–5 tokens
- **Myth:** "Tokenizers are the same across models." Each model family trains its own tokenizer
- **Myth:** "Tokenization does not affect model intelligence." Tokenization directly shapes what patterns are learnable
- **Myth:** "Token limits are a hardware constraint." They are primarily a training sequence length constraint



Key Takeaways

What You Have Learned

- Tokenization converts raw text into sequences of integer IDs that neural networks can process
- Modern LLMs use **subword tokenization** (BPE, WordPiece, SentencePiece) for the best vocabulary efficiency
- The tokenizer builds a fixed **vocabulary** mapping strings to IDs; special tokens enable chat and masking
- Tokens are the unit of cost and context for all commercial LLM APIs
- Tokenization edge cases (numbers, code, non-English text) explain many apparent model limitations
- The three major algorithms — BPE, WordPiece, SentencePiece — are covered in subsequent concepts

Thank You

What is Tokenization? — Module 3.3